

AI Cost Intelligence & Autonomous Action System

Detailed Project Documentation

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. System Architecture
- 4. Technology Stack
- 5. Project Structure
- 6. Data Models
- 7. Multi-Agent Architecture
- 8. Workflow Engine
- 9. Enterprise Approval Workflow
- 10. Audit Logging System
- 11. REST API Reference
- 12. Frontend Dashboard
- 13. Sample Data Design
- 14. Financial Impact Calculation
- 15. How to Run
- 16. Future Enhancements

1. Project Overview

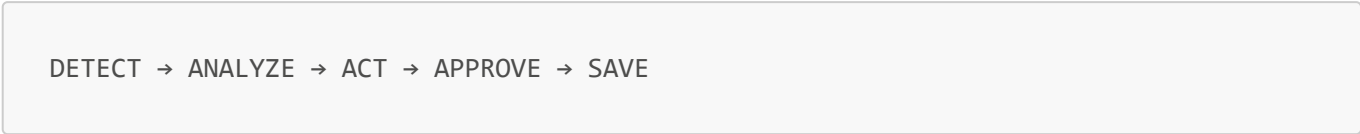
The **AI Cost Intelligence & Autonomous Action System** is an enterprise-grade, full-stack application that uses a **multi-agent architecture** to automatically detect financial inefficiencies, initiate corrective actions, and generate measurable cost savings. Unlike traditional dashboards that only display data, this system goes beyond reporting — it **autonomously detects, analyzes, acts, and tracks** financial waste across an organization.

Key Capabilities

- **Automated Inefficiency Detection** — Four specialized AI agents continuously scan enterprise transaction data for anomalies, duplicate payments, SLA breaches, underutilized resources, and billing discrepancies.
- **Autonomous Corrective Actions** — The system doesn't just flag issues; it initiates real corrective actions like payment reversals, resource reallocation, vendor renegotiation, and SLA escalation.
- **Actionable Playbooks with Cost-Math** — For every detected issue, the system generates a step-by-step remediation playbook with explicit financial calculations showing the expected savings.
- **Enterprise Approval Workflows** — High-risk and high-value actions are gated behind an approval queue where human decision-makers can approve or reject proposed changes.
- **Full Audit Trail** — Every agent action, detection, and decision is logged with timestamps for compliance and accountability.

- **Real-Time Dashboard** — A premium dark-mode dashboard with interactive charts, KPI cards, and real-time data visualization.

Core Pipeline



Each agent follows this five-stage pipeline:

1. **DETECT** — Scan transaction data for issues
2. **ANALYZE** — Confirm and prioritize detected issues, escalate severity
3. **ACT** — Generate corrective action items (auto-execute or queue for approval)
4. **APPROVE** — High-risk actions enter the enterprise approval queue
5. **SAVE** — Calculate and report financial impact and ROI

2. Problem Statement

Build an AI system that goes beyond dashboards. It should continuously monitor enterprise operations data, identify cost leakage or inefficiency patterns, and initiate corrective actions with quantifiable financial impact.

What You May Build

- **Spend intelligence agents** — that dig into procurement, vendor, and operations data to find anomalies, duplicate costs, and rate optimization opportunities — then generate actionable playbooks or trigger downstream workflows.
- **Service Level Agreement (SLA) and penalty prevention agents** — that detect approaching breaches from operational signals and reroute work, shift resources, or escalate before the financial hit lands.
- **Resource optimization agents** — that monitor utilization across tools, infrastructure, and teams — recommending consolidation and executing approved changes.
- **Financial operations agents** — that reconcile transactions, flag discrepancies, and produce variance analyses with root-cause attribution to cut close cycles.

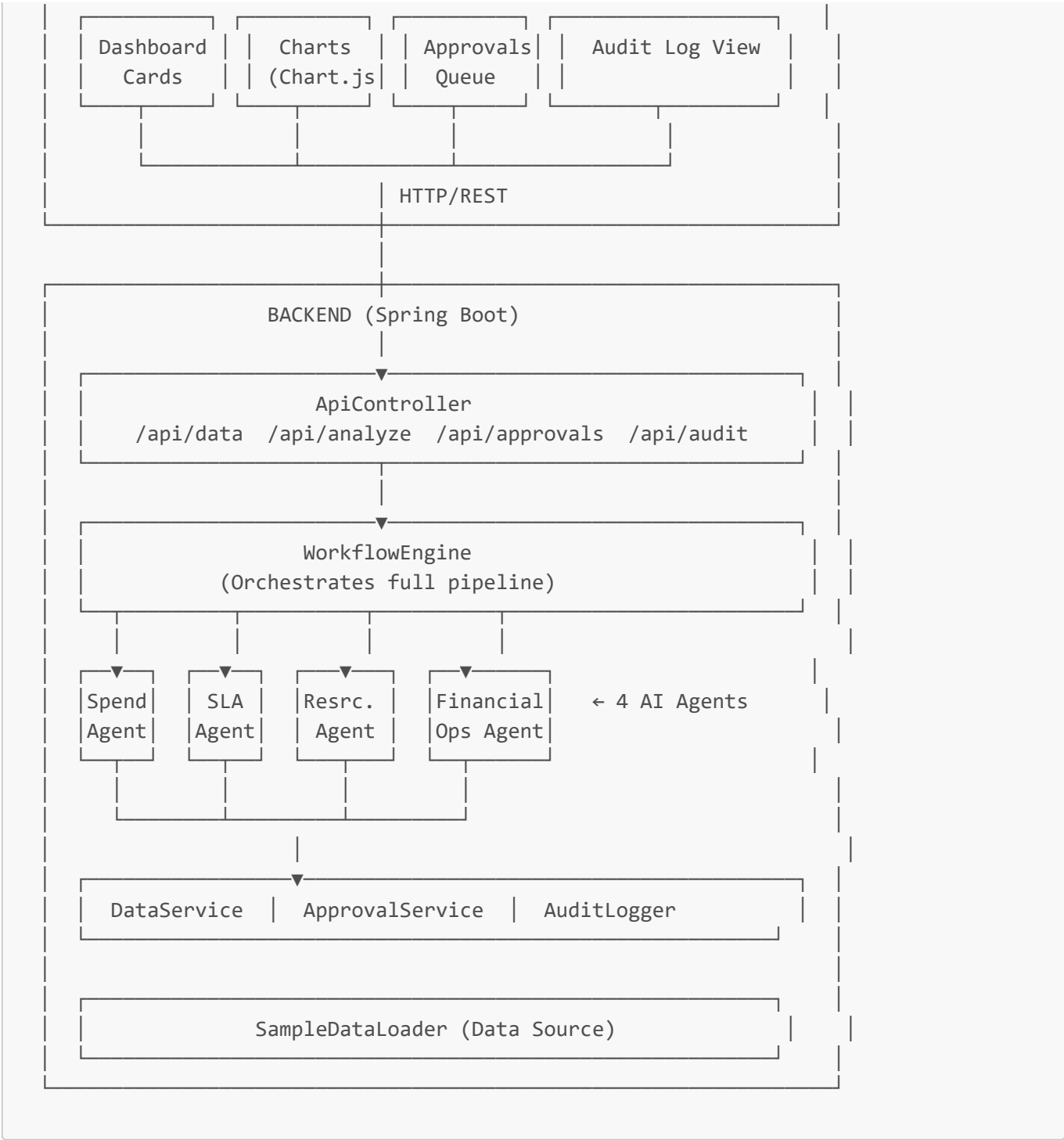
Evaluation Focus

- Quantifiable cost impact (show the math)
- Ability to take action — not just generate reports
- Data integration depth
- Ability to work within enterprise approval workflows

3. System Architecture

High-Level Architecture Diagram





Architectural Patterns Used

Pattern	Description
Multi-Agent Architecture	Four autonomous agents operate independently with a common interface
Strategy Pattern	Each agent implements the Agent interface with its own detection/action logic
Pipeline Pattern	WorkflowEngine orchestrates agents in a sequential detect → analyze → act pipeline
Service Layer	Business logic separated into DataService, ApprovalService, and WorkflowEngine

Pattern	Description
MVC	Spring Boot controller handles HTTP requests, services manage business logic, models represent data
Observer Pattern	AuditLogger records events emitted by all agents and services

4. Technology Stack

Backend

Technology	Version	Purpose
Java	17	Programming language
Spring Boot	3.2.3	Application framework
Spring Web	(starter)	REST API and HTTP server
Maven	(wrapper)	Build tool and dependency management

Frontend

Technology	Version	Purpose
HTML5	—	Page structure and semantic markup
CSS3	—	Styling with custom dark theme, glassmorphism, animations
JavaScript	ES6+	Dynamic rendering, API calls, chart rendering
Chart.js	4.4.1	Interactive data visualizations (bar, doughnut, horizontal bar charts)
Inter Font	(Google Fonts)	Modern typography

Build & Configuration

File	Purpose
<code>pom.xml</code>	Maven project configuration with Spring Boot parent
<code>application.properties</code>	Server port (8080) and app name configuration
<code>mvnw / mvnw.cmd</code>	Maven wrapper for consistent builds

5. Project Structure

ET/	
├─ pom.xml	# Maven configuration
├─ mvnw / mvnw.cmd	# Maven wrapper
└─ frontend/	# Frontend application

├── index.html	# Main HTML page (291 lines)
├── styles.css	# Complete CSS styling (22KB)
└── app.js	# Dashboard logic (601 lines)
└── src/main/java/com/costintel/	
├── CostIntelApplication.java	# Spring Boot entry point
├── agents/	# AI Agent implementations
├── Agent.java	# Agent interface (contract)
├── SpendIntelligenceAgent.java	# Spend analysis agent (263 lines)
├── SLAMonitoringAgent.java	# SLA monitoring agent (200 lines)
├── ResourceOptimizationAgent.java	# Resource optimization agent (178
lines)	
└── FinancialOperationsAgent.java	# Financial operations agent (188
lines)	
├── config/	
└── WebConfig.java	# CORS and static resource
configuration	
├── controller/	
└── ApiController.java	# REST API endpoints (136 lines)
├── models/	# Data models
├── Transaction.java	# Transaction data model
├── Issue.java	# Detected issue model
├── ActionItem.java	# Corrective action model
├── Playbook.java	# Remediation playbook model
├── ApprovalRequest.java	# Approval queue item model
├── AuditEntry.java	# Audit log entry model
└── Impact.java	# Financial impact summary model
├── service/	# Business logic services
├── WorkflowEngine.java	# Pipeline orchestrator (176 lines)
├── DataService.java	# Data management service
└── ApprovalService.java	# Approval queue management
└── utils/	# Utility classes
├── AuditLogger.java	# Thread-safe audit logging
└── SampleDataLoader.java	# Enterprise sample data generator

6. Data Models

6.1 Transaction

The core data entity representing an enterprise financial transaction.

Field	Type	Description
id	String	Unique identifier (e.g., TXN-001)

Field	Type	Description
vendor	String	Vendor/supplier name
amount	double	Transaction amount in ₹
date	String	Transaction date (yyyy-MM-dd)
resourceUsage	double	Resource utilization percentage (0-100)
expectedAmount	double	Expected/budgeted amount
actualAmount	double	Actual invoiced amount
category	String	Business category (Infrastructure, Software, etc.)
status	String	Transaction status (COMPLETED, PENDING, ACTIVE, IN_PROGRESS)
deadline	String	SLA deadline date (yyyy-MM-dd)

Design Decision: The Transaction model is intentionally rich with fields supporting all four agents' detection logic — `resourceUsage` for the Resource agent, `expectedAmount`/`actualAmount` for the Financial agent, `deadline` for the SLA agent, and `vendor`/`amount`/`date` for the Spend agent.

6.2 Issue

Represents a detected financial issue or inefficiency.

Field	Type	Description
id	String	Unique issue ID (e.g., <code>SI-1</code> , <code>SLA-2</code> , <code>RO-3</code> , <code>FO-4</code>)
type	String	Issue classification (see table below)
severity	String	CRITICAL / HIGH / MEDIUM / LOW
description	String	Human-readable description with financial details
agent	String	Name of the detecting agent
affectedTransactionId	String	Related transaction ID (nullable)
suggestedAction	String	Recommended corrective action
potentialSavings	double	Estimated recoverable amount in ₹

Issue Types by Agent:

Agent	Issue Types
Spend Intelligence	<code>DUPLICATE_PAYMENT</code> , <code>SPENDING_ANOMALY</code> , <code>RECURRING_EXPENSE</code> , <code>RATE_OPTIMIZATION</code>
SLA Monitoring	<code>SLA_BREACH</code> , <code>SLA_AT_RISK</code> , <code>SLA_RESOURCE_RISK</code>
Resource Optimization	<code>UNDERUTILIZED_RESOURCE</code> , <code>CONSOLIDATION_OPPORTUNITY</code>

Agent	Issue Types
Financial Operations	TRANSACTION_MISMATCH, SYSTEMIC_OVERBILLING

6.3 ActionItem

Represents a corrective action taken or proposed.

Field	Type	Description
id	String	Unique action ID (e.g., ACT-SI-1)
type	String	Action classification (e.g., FLAG_DUPLICATE, ESCALATE_AND_REROUTE)
description	String	Detailed description of the action taken
status	String	EXECUTED, PENDING_APPROVAL, PENDING_REVIEW, IN_PROGRESS
savings	double	Associated savings amount in ₹
relatedIssueId	String	The issue that triggered this action

Action Statuses:

- EXECUTED — Action was automatically executed (low-risk)
- PENDING_APPROVAL — Action requires human approval (high-risk/high-value)
- PENDING_REVIEW — Action needs management review
- IN_PROGRESS — Action is currently being executed

6.4 Playbook

An actionable remediation playbook with step-by-step instructions and cost-math.

Field	Type	Description
id	String	Unique playbook ID (e.g., PB-SI-1)
title	String	Descriptive title
agent	String	Generating agent name
triggerIssueId	String	The issue that triggered playbook generation
steps	List<String>	Ordered remediation steps (3-7 steps)
costMath	String	Explicit financial calculation with formulas
estimatedSavings	double	Total estimated savings in ₹
status	String	GENERATED, APPROVED, EXECUTED, REJECTED
priority	String	CRITICAL, HIGH, MEDIUM, LOW

Sample Cost-Math Output:

--

Duplicate Amount: ₹5,000 Recovery Rate: 100% Net Savings: ₹5,000		
Current Premium: ₹16,667 Target Reduction: 30% Projected Savings: ₹5,000		
Post-Consolidation Cost: ₹3,500 Migration Cost: ~₹820 Net Annual Savings: ₹88,506		
Mismatch Amount: ₹5,000 Recovery Rate: 95% Expected Recovery: ₹4,750 Close Cycle Reduction: ~2 days		

6.5 ApprovalRequest

Represents a pending approval in the enterprise approval queue.

Field	Type	Description
id	String	Unique approval ID (e.g., APR-1)
playbookId	String	Linked playbook ID (nullable)
agent	String	Requesting agent name
description	String	Human-readable description of the proposed action
riskLevel	String	HIGH (savings > ₹5,000) or MEDIUM
estimatedImpact	double	Financial impact of the action in ₹
status	String	PENDING, APPROVED, REJECTED
createdAt	String	Timestamp when created
resolvedAt	String	Timestamp when approved/rejected (nullable)

6.6 AuditEntry

Immutable audit log entry for compliance tracking.

Field	Type	Description
timestamp	String	Event timestamp (yyyy-MM-dd HH:mm:ss)
step	int	Sequential step number in the pipeline
agent	String	Agent or system component name
message	String	Human-readable event description

6.7 Impact

Aggregated financial impact summary.

Field	Type	Description
totalCost	double	Sum of all transaction amounts
totalWaste	double	Sum of all detected waste/savings potential

Field	Type	Description
totalSavings	double	Sum of all action-derived savings
monthlySavings	double	Monthly savings (equals totalSavings)
yearlySavings	double	Yearly projection (totalSavings × 12)
wasteBreakdown	Map<String, Double>	Waste amount grouped by issue type

7. Multi-Agent Architecture

7.1 Agent Interface

All agents implement a common `Agent` interface, ensuring consistent behavior:

```
public interface Agent {
    String getName();
    List<Issue> detect(List<Transaction> transactions);
    List<Issue> analyze(List<Issue> issues);
    List<ActionItem> act(List<Issue> issues);

    default List<Playbook> generatePlaybooks(List<Issue> issues) {
        return new ArrayList<>();
    }
}
```

Each agent independently handles:

- **Detection** — Scanning transaction data for specific patterns
- **Analysis** — Confirming and re-prioritizing detected issues
- **Action** — Generating corrective actions with appropriate statuses
- **Playbook Generation** — Creating step-by-step remediation guides with cost-math

7.2 Spend Intelligence Agent

Purpose: Digs into procurement, vendor, and operations data to find anomalies, duplicate costs, and rate optimization opportunities — then generates actionable playbooks or triggers downstream workflows.

Detection Algorithms:

Detection	Algorithm	Threshold
Duplicate Payments	Groups transactions by <code>vendor + amount + date</code> ; flags groups with size > 1	Any duplicate
Spending Anomalies	Calculates per-vendor average; flags transactions > 2× average	Amount > ₹1,000 and > 2× avg

Detection	Algorithm	Threshold
Recurring Expenses	Counts transactions per vendor; flags vendors with ≥ 3 payments	Total > ₹5,000
Rate Optimization	Compares vendor prices within the same category	Max > $1.5 \times$ min and max > ₹5,000

Actions Generated:

Issue Type	Action Type	Status	Description
Duplicate Payment	FLAG_DUPLICATE	EXECUTED	Flags and initiates reversal process
Spending Anomaly	ALERT_ANOMALY	PENDING_REVIEW	Triggers management review alert
Recurring Expense	REVIEW_SUBSCRIPTION	EXECUTED	Suggests cancellation review
Rate Optimization	NEGOTIATE_RATE	PENDING_APPROVAL	Initiates vendor renegotiation workflow

Playbook Examples:

1. **Reverse Duplicate Payment**

— 5-step playbook: Verify in ERP → Flag transaction → Initiate reversal → Notify vendor → Update ledger
2. **Investigate Spending Spike**

— 5-step playbook: Pull history → Compare amounts → Request justification → Cap future spend → Set alerts
3. **Vendor Rate Renegotiation**

— 5-step playbook: Benchmark rates → Get competitive quotes → Prepare negotiation brief → Schedule meeting → Execute new contract

7.3 SLA Monitoring Agent

Purpose: Detects approaching SLA breaches from operational signals and reroutes work, shifts resources, or escalates before the financial hit lands.

Detection Logic:

Detection	Condition	Severity
SLA Breach	Deadline passed ($\text{daysUntil} < 0$)	CRITICAL
SLA At Risk	Deadline within 3 days ($\text{daysUntil} \leq 3$)	HIGH
SLA Resource Risk	Deadline within 7 days AND resource usage < 50%	MEDIUM

Financial Calculations:

- SLA Breach Penalty: $\text{transaction amount} \times 10\%$
- SLA At Risk Exposure: $\text{transaction amount} \times 5\%$

- Resource Risk Exposure: $\text{transaction amount} \times 3\%$

Actions Generated:

Issue Type	Action Type	Status	Description
SLA Breach	ESCALATE_AND_REROUTE	EXECUTED	Escalates breach, activates penalty clause, reroutes work to backup vendor
SLA At Risk	SHIFT_RESOURCES	IN_PROGRESS	Allocates additional team, sends urgent stakeholder notifications
SLA Resource Risk	INCREASE_ALLOCATION	PENDING_APPROVAL	Submits resource reallocation request, initiates workload redistribution

Playbook Examples:

1. **SLA Breach Recovery & Penalty Mitigation** — 6-step playbook: Document breach → Activate penalty clause → Reroute deliverables → Negotiate penalty reduction → Update dashboard → Schedule post-mortem
2. **Prevent SLA Breach — Emergency Resource Shift** — 5-step playbook: Assess progress → Allocate resources → Notify vendor → Set hourly tracking → Prepare contingency

7.4 Resource Optimization Agent

Purpose: Monitors utilization across tools, infrastructure, and teams — recommending consolidation and executing approved changes.

Detection Logic:

Detection	Condition	Severity
Underutilized Resource	$\text{resourceUsage} > 0 \text{ and } < 40\%$	HIGH if $< 20\%$, MEDIUM otherwise
Consolidation Opportunity	Multiple underutilized resources in same category	HIGH (CRITICAL if savings $> ₹10,000$)

Financial Calculations:

- Wasted Portion: $(1 - \text{resourceUsage}/100) \times \text{amount}$
- Consolidation Savings: $\text{totalCost} \times (1 - \text{avgUsage}/100) \times 0.7$

Actions Generated:

Issue Type	Action Type	Status	Description
Underutilized Resource	OPTIMIZE_RESOURCE	EXECUTED ($< ₹5K$) or PENDING_APPROVAL ($\geq ₹5K$)	Right-size recommendation

Issue Type	Action Type	Status	Description
Consolidation Opportunity	CONSOLIDATE_RESOURCES	PENDING_APPROVAL	Resource consolidation workflow

Playbook Examples:

1. **Resource Consolidation Execution** — 7-step playbook: Audit utilization → Design consolidated architecture → Submit plan for review → Execute migration → Decommission redundant instances → Verify service levels → Update billing
2. **Right-Size Underutilized Resource** — 5-step playbook: Review 30-day metrics → Identify right-sized tier → Schedule downgrade → Execute and monitor → Confirm cost reduction

7.5 Financial Operations Agent

Purpose: Reconciles transactions, flags discrepancies, and produces variance analyses with root-cause attribution to cut close cycles.

Detection Logic:

Detection	Condition	Severity
Transaction Mismatch	<code>expectedAmount</code> $\neq$ <code>actualAmount</code> (diff > ₹0.01)	CRITICAL (> ₹5K), HIGH (> ₹1K), MEDIUM
Systemic Overbilling	Vendor's average variance > 3% across $\geq$ 2 transactions	HIGH

Root-Cause Attribution:

Variance	Root Cause Classification
> 15%	CONTRACT_DEVIATION
> 5%	PRICING_ERROR
$\leq$ 5%	QUANTITY_DISCREPANCY

Actions Generated:

Issue Type	Action Type	Status	Description
Transaction Mismatch	RECONCILE_TRANSACTION	EXECUTED (< ₹3K) or PENDING_APPROVAL ($\geq$ ₹3K)	Financial reconciliation with refund request
Systemic Overbilling	VENDOR_AUDIT	PENDING_APPROVAL	Full vendor audit and contract review

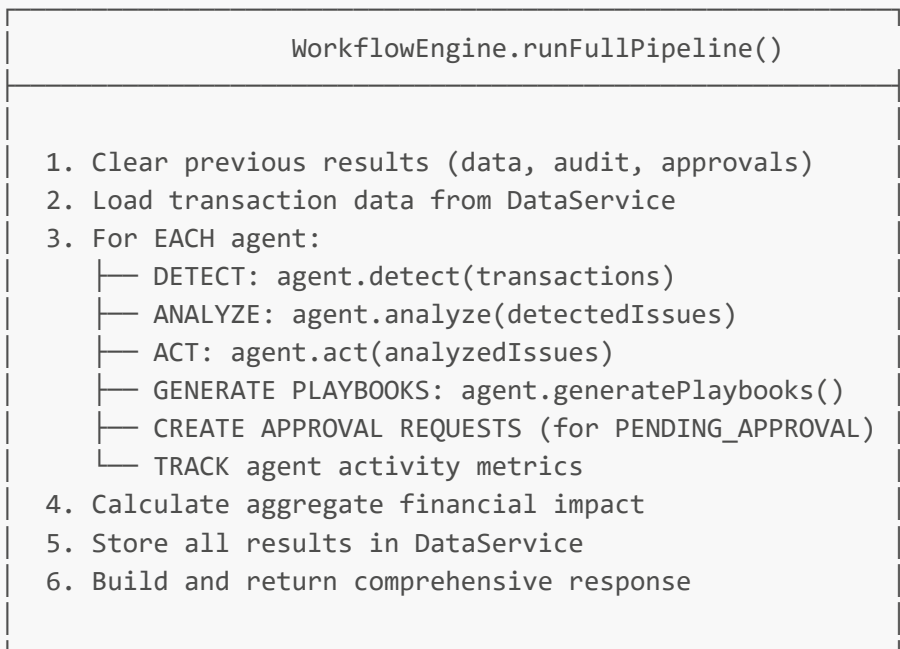
Playbook Examples:

1. **Transaction Reconciliation & Recovery** — 5-step playbook: Verify PO/invoice → Identify root cause → Submit refund request → Update ledger → Flag vendor for monitoring
 2. **Vendor Audit & Contract Renegotiation** — 7-step playbook: Pull 12-month history → Calculate cumulative overcharge → Cross-reference contract → Prepare audit report → Schedule vendor meeting → Negotiate credit note → Amend contract
-

8. Workflow Engine

The **WorkflowEngine** is the central orchestrator of the entire system. It coordinates all agents through the detection pipeline and produces the final analysis result.

Pipeline Execution Flow



Approval Request Auto-Generation

The WorkflowEngine automatically creates approval requests for high-risk actions:

- Scans all generated action items for **status** == "PENDING_APPROVAL"
- Assigns **risk level** based on savings amount:
 - **HIGH** if savings > ₹5,000
 - **MEDIUM** otherwise
- Links the approval to the corresponding playbook via **findPlaybookForIssue()**
- Adds the request to the **ApprovalService** queue

Financial Impact Calculation

```
// Total cost = sum of all transaction amounts
totalCost = Σ transaction.getAmount()
```

```
// Total waste = sum of all detected issue savings potential
totalWaste =  $\Sigma$  issue.getPotentialSavings()

// Total savings = sum of all corrective action savings
totalSavings =  $\Sigma$  action.getSavings()

// Monthly savings = total savings (one analysis run = one month)
monthlySavings = totalSavings

// Yearly projection = monthly  $\times$  12
yearlySavings = totalSavings  $\times$  12

// Waste breakdown = savings grouped by issue type
wasteBreakdown = GROUP BY issue.type  $\rightarrow$  SUM(potentialSavings)
```

9. Enterprise Approval Workflow

The approval system ensures that high-risk, high-value actions are reviewed by humans before execution.

Flow

```
Agent detects issue  $\rightarrow$  Action created with PENDING_APPROVAL
|
▼
WorkflowEngine creates ApprovalRequest
|
▼
ApprovalService stores in queue
|
▼
Dashboard shows pending approvals with Approve/Reject buttons
|
├── User clicks "Approve"  $\rightarrow$  POST /api/approvals/{id}/approve
│    $\rightarrow$  status = APPROVED, resolvedAt = now
│    $\rightarrow$  Audit log: "Approved request APR-1: ..."
|
└── User clicks "Reject"  $\rightarrow$  POST /api/approvals/{id}/reject
     $\rightarrow$  status = REJECTED, resolvedAt = now
     $\rightarrow$  Audit log: "Rejected request APR-1: ..."
```

Key Design Decisions

- **Risk-based gating:** Only actions with **PENDING_APPROVAL** status enter the queue (not **EXECUTED** or **IN_PROGRESS**)
- **Financial threshold:** Actions over ₹5,000 are tagged **HIGH** risk, others as **MEDIUM**
- **Audit integration:** Every approval/rejection decision is automatically logged
- **Idempotent operations:** Can only approve/reject **PENDING** requests (prevents double-processing)

10. Audit Logging System

Implementation

The `AuditLogger` is a thread-safe, centralized logging service:

```
@Component
public class AuditLogger {
    private final List<AuditEntry> auditLog = Collections.synchronizedList(new
ArrayList<>());
    private final AtomicInteger stepCounter = new AtomicInteger(0);
}
```

What Gets Logged

Event	Source	Example Message
Data loading	System	Data loaded: 25 transactions
Detection start	Agent	Starting detection phase
Issues detected	Agent	Detected 4 issues
Analysis complete	Agent	Analysis complete. 4 issues confirmed
Actions executed	Agent	Executed 4 corrective actions
Playbooks generated	Agent	Generated 4 actionable playbooks
Approvals queued	Agent	2 actions queued for approval
Impact calculated	System	Calculating financial impact
Pipeline complete	System	Pipeline complete. Total savings: ₹85,000 \\\nPlaybooks: 12 \\\nPending approvals: 5
Approval decisions	Approval System	Approved request APR-1: ...

11. REST API Reference

Base URL: `http://localhost:8080/api`

Method	Endpoint	Description	Response
GET	/api/data	Get all transactions	List<Transaction>
POST	/api/analyze	Run full analysis pipeline	Full pipeline result (JSON)
GET	/api/results	Get detected issues and actions	{issues, actions}
GET	/api/impact	Get financial impact summary	Impact
GET	/api/audit	Get audit log entries	List<AuditEntry>
GET	/api/agents	Get agent activity metrics	Map<String, AgentActivity>
GET	/api/actions	Get all corrective actions	List<ActionItem>
GET	/api/playbooks	Get all generated playbooks	List<Playbook>
GET	/api/approvals	Get all approval requests	List<ApprovalRequest>
POST	/api/approvals/{id}/approve	Approve a pending request	ApprovalRequest
POST	/api/approvals/{id}/reject	Reject a pending request	ApprovalRequest
GET	/api/dashboard/summary	Get dashboard summary with ROI metrics	Dashboard summary (JSON)

CORS Configuration

All `/api/**` endpoints allow cross-origin requests from any origin (*) with methods GET, POST, PUT, DELETE, and OPTIONS.

Sample `/api/analyze` Response Structure

```
{
  "totalIssues": 15,
  "totalActions": 15,
  "totalPlaybooks": 15,
  "pendingApprovals": 5,
  "impact": {
    "totalCost": 399000,
    "totalWaste": 65000,
    "totalSavings": 52000,
    "monthlySavings": 52000,
    "yearlySavings": 624000,
    "wasteBreakdown": {
      "DUPLICATE_PAYMENT": 13500,
      "SLA_BREACH": 3800,
      "UNDERUTILIZED_RESOURCE": 28000,
      "TRANSACTION_MISMATCH": 7500
    }
  }
}
```

```
  },
  "agentActivity": { ... },
  "actions": [ ... ],
  "playbooks": [ ... ],
  "approvals": [ ... ],
  "auditLog": [ ... ]
}
```

12. Frontend Dashboard

Dashboard Sections

The frontend is a **premium dark-mode dashboard** built with vanilla HTML/CSS/JS. It consists of the following sections:

Section	Description
Header	Logo, pipeline badge (DETECT → ANALYZE → ACT → APPROVE → SAVE), Run Analysis button
Summary Cards	4 KPI cards: Total Cost, Waste Detected, Savings Generated, Yearly Projection
ROI Metrics	6 metric cards: Efficiency %, Actions Executed, Pending Approvals, Playbooks Generated, Cost Recovered, Pending Savings
Charts Row	Bar chart (Cost vs Savings Analysis) and Doughnut chart (Waste Distribution)
Agent Performance	Horizontal bar chart comparing all 4 agents (issues, actions, playbooks)
Transactions Table	All 25 transactions with usage bars, amounts, and status badges
Detected Issues Table	Issues with severity badges, descriptions, agent attribution, savings
Corrective Actions Table	Actions with type badges, status badges, savings
Actionable Playbooks	Expandable playbook cards with steps and cost-math
Enterprise Approval Queue	Approval cards with risk badges and Approve/Reject buttons
Agent Activity	Per-agent statistics (issues, actions, playbooks, savings)
Audit Log	Scrollable timeline of all pipeline events

Design Features

- **Dark Mode** — Deep dark background (#0a0b14) with glassmorphism panels

- **Animated Values** — Summary card values animate from 0 to target with cubic easing (1200ms)
- **Color-Coded Severity** — CRITICAL (rose), HIGH (amber), MEDIUM (indigo), LOW (slate)
- **Interactive Charts** — Chart.js with custom tooltips, gradient colors, and smooth animations
- **Responsive Usage Bars** — Color-coded resource utilization bars (red < 40%, amber < 70%, green ≥ 70%)
- **Expandable Playbooks** — Click to expand/collapse with chevron rotation
- **Real-Time Approvals** — Approve/Reject buttons trigger API calls and refresh the queue
- **Indian Number Formatting** — Currency displayed as ₹X,XXX or ₹X.XXL for lakhs

13. Sample Data Design

The **SampleDataLoader** generates **25 transactions** specifically crafted to trigger all detection algorithms across all four agents:

Transaction IDs	Purpose	Triggers
TXN-001 to TXN-003	Normal baseline transactions	Reference data for averages
TXN-004, TXN-005	Duplicate payments (Office Supplies Co, ₹5,000, same date)	Spend Agent: DUPLICATE_PAYMENT
TXN-006, TXN-007	Duplicate payments (Tech Solutions Ltd, ₹8,500, same date)	Spend Agent: DUPLICATE_PAYMENT
TXN-008, TXN-009	SLA breaches (past deadline by 5 days)	SLA Agent: SLA_BREACH
TXN-010	SLA at risk (deadline in 2 days)	SLA Agent: SLA_AT_RISK
TXN-011	SLA resource risk (15% utilization, deadline in 6 days)	SLA Agent: SLA_RESOURCE_RISK
TXN-012, TXN-013, TXN-025	Underutilized resources (10-22% usage, Infrastructure)	Resource Agent: UNDERUTILIZED_RESOURCE + CONSOLIDATION_OPPORTUNITY
TXN-014	Underutilized resource (35% usage, Storage)	Resource Agent: UNDERUTILIZED_RESOURCE
TXN-015, TXN-018	Financial mismatch — overcharge (Marketing Agency)	Financial Agent: TRANSACTION_MISMATCH + SYSTEMIC_OVERBILLING
TXN-016	Financial mismatch — overcharge (Logistics Partner, variance ~11%)	Financial Agent: TRANSACTION_MISMATCH
TXN-017	Financial mismatch — undercharge (Equipment Vendor)	Financial Agent: TRANSACTION_MISMATCH
TXN-019, TXN-020	Spending anomaly (AWS spike from ₹3K avg to ₹55K)	Spend Agent: SPENDING_ANOMALY

Transaction IDs	Purpose	Triggers
TXN-021, TXN-022 + TXN-002	Recurring expenses (Google Workspace ×3, total ₹15K)	Spend Agent: RECURRING_EXPENSE
TXN-023	Rate comparison (Budget Consulting ₹3.5K vs TXN-006 ₹8.5K in Consulting)	Spend Agent: RATE_OPTIMIZATION
TXN-024	Rate comparison (Premium Analytics ₹35K vs TXN-008 ₹20K in Analytics)	Spend Agent: RATE_OPTIMIZATION

14. Financial Impact Calculation

How Savings Are Calculated Per Issue Type

Issue Type	Savings Formula	Example
Duplicate Payment	duplicate transaction amount (100% recovery)	₹5,000 duplicate → ₹5,000 savings
Spending Anomaly	amount - vendor average	₹55K spent, avg ₹24.3K → ₹30.7K excess
Recurring Expense	total × 15% (estimated reduction)	3 payments totaling ₹15K → ₹2,250 savings
Rate Optimization	(max - min) × 30% (negotiation target)	₹8.5K vs ₹3.5K spread → ₹1,500 savings
SLA Breach	transaction amount × 10% (penalty recovery)	₹20K contract → ₹2,000 penalty savings
SLA At Risk	transaction amount × 5% (risk avoided)	₹25K contract → ₹1,250 risk avoided
SLA Resource Risk	transaction amount × 3% (risk avoided)	₹12K contract → ₹360 risk avoided
Underutilized Resource	(1 - usage/100) × amount	15% usage on ₹12K → ₹10,200 wasted
Consolidation	totalCost × (1 - avgUsage/100) × 0.7	Multiple resources, 70% recovery
Transaction Mismatch (Overcharge)	abs(expected - actual)	Expected ₹30K, actual ₹35K → ₹5,000
Systemic Overbilling	0 (flagged for audit, savings TBD)	Flagged for manual audit

Severity Escalation Rules

Agent	Escalation Rule
-------	-----------------

Agent	Escalation Rule
Spend Intelligence	savings > ₹10K → CRITICAL; > ₹5K → HIGH
SLA Monitoring	SLA_BREACH → always CRITICAL
Resource Optimization	savings > ₹8K → HIGH; CONSOLIDATION + > ₹10K → CRITICAL
Financial Operations	savings > ₹5K → CRITICAL

15. How to Run

Prerequisites

- **Java 17+** (JDK) installed
- **Maven** (or use the included Maven wrapper)
- A modern web browser (Chrome, Firefox, Edge)

Steps

1. **Clone/Download the project** to your local machine.
2. **Start the Spring Boot backend:**

```
# Using Maven wrapper (no Maven installation needed)
./mvnw spring-boot:run

# Or on Windows
mvnw.cmd spring-boot:run
```

The server starts on <http://localhost:8080>.

3. **Open the frontend dashboard:**
 - Open [frontend/index.html](#) in your browser
 - Or serve it via any static file server
4. **Run the analysis:**
 - Click the **"Run Analysis"** button on the dashboard
 - The system will:
 - Load 25 sample transactions
 - Run all 4 agents through the full pipeline
 - Display results on the dashboard
5. **Interact with the system:**
 - Review detected issues in the Issues table
 - Explore actionable playbooks (click to expand)
 - Approve or reject actions in the Approval Queue

- View the audit log for full pipeline traceability
- Examine charts for financial impact visualization

16. Future Enhancements

Enhancement	Description
Continuous Monitoring	Add @Scheduled cron jobs for automatic periodic analysis
Real Data Integration	Connect to databases, ERP systems, cloud billing APIs (AWS Cost Explorer, Azure Cost Management)
Machine Learning	Replace rule-based detection with ML anomaly detection models
Notifications	Email/Slack alerts for critical issues and approval requests
Role-Based Access	Different approval authority levels (manager, director, VP)
Database Persistence	Replace in-memory storage with PostgreSQL/MongoDB
Historical Trending	Track savings over time with trend charts
Multi-Tenant Support	Support multiple organizations/departments

Document Version: 2.0 Last Updated: March 2026 System Version: AI Cost Intelligence v2.0 — Multi-Agent Autonomous Action Platform